Universidad Tecnológica Nacional

Desarrollo de Software Cloud

TicketLite Cloud

Entrega 2

Muñoz Joaquin

1. En qué consiste el proyecto

TicketLite Cloud es un sistema de gestión de incidencias (tickets) planteado para equipos de trabajo chicos o medianos que necesitan organizar solicitudes internas sin depender de herramientas pagas o sobredimensionadas. Combina una aplicación web desarrollada en Django con servicios gestionados de AWS para el envío asíncrono de notificaciones y con un servicio de IA que sugiere la prioridad inicial de cada ticket.

El sistema tiene dos puertas de entrada: el equipo interno, que trabaja autenticado, y un canal público donde cualquier persona puede reportar un problema sin tener cuenta y seguir el estado de su ticket mediante un código de seguimiento.

Funcionalidades principales

Área	Descripción
Gestión de tickets	Creación, listado y detalle de tickets con estado (Pendiente, En proceso, Resuelto), prioridad (Baja, Media, Alta) y responsable asignado. Comentarios, historial de cambios de estado, dashboard con métricas, y filtros, búsqueda, orden y paginación en los listados.
Roles y permisos	Los usuarios autenticados crean tickets y pueden autoasignarse los que no tienen responsable. Solo el responsable o un administrador puede cambiar estado y prioridad. Los administradores gestionan usuarios.
Notificaciones asíncronas	Cada cambio de estado publica un evento que viaja por SQS, Lambda y SNS hasta llegar como email a la casilla suscripta al topic de SNS.
Prioridad inicial con IA	Al crear un ticket, la API de Gemini analiza el ticket y sugiere la prioridad. Si la IA no responde a tiempo o falla, el ticket se crea igual con prioridad Media.
Reporte público	Una persona sin cuenta puede reportar un problema, obtiene un código de seguimiento único y puede consultar con él el estado de ese ticket, y solo de ese. Una vez reportado, el ticket aparece en el sistema interno y se gestiona como cualquier otro.

2. Herramientas y tecnologías

Componente	Tecnología	Para qué se usa
Aplicación	Python, Django, Bootstrap 5	Lógica de negocio e interfaz web con templates del lado del servidor.
Base de datos	PostgreSQL (Neon en producción, contenedor Docker en local)	Persistencia de tickets, comentarios, historial y usuarios.
Deploy	Render	Despliegue de la aplicación como PaaS y servicio de archivos estáticos.

Componente	Tecnología	Para qué se usa
Contenedores	Docker y Docker Compose	Entorno de desarrollo local con la aplicación y la base de datos.
Mensajería	Amazon SQS	Cola que desacopla el evento de negocio del envío de la notificación.
Cómputo serverless	AWS Lambda (Python)	Procesa cada mensaje de la cola sin mantener un servidor corriendo.
Notificaciones	Amazon SNS	Envío del email de aviso.
Observabilidad	Amazon CloudWatch	Logs de ejecución de la función Lambda.
Acceso a AWS	AWS IAM	Usuario dedicado con permisos acotados, en lugar de las credenciales root.
IA	API de Gemini	Sugerencia de la prioridad inicial de cada ticket.
Calidad	Tests de Django	Suite de pruebas automatizadas, con las llamadas a AWS y a la IA simuladas.
Control de versiones	Git y GitHub	Historial de commits con la convención Conventional Commits.

El flujo de notificaciones es el siguiente: cuando cambia el estado de un ticket, un signal de Django publica un mensaje en SQS usando las credenciales del usuario IAM. La cola dispara una función Lambda que publica el aviso en SNS, y SNS envía el email. La prioridad inicial, en cambio, se resuelve con una llamada síncrona a Gemini desde la vista de creación de tickets.

3. Decisiones de diseño

Decisión	Motivo
Django monolítico, sin frontend separado	Para el alcance de un sistema interno de un equipo chico, mantener dos aplicaciones no aporta un beneficio que justifique la complejidad. La complejidad cloud-native está en el desacoplamiento por eventos.
IA mediante API externa, no Bedrock	Una llamada síncrona dentro de la creación del ticket simplifica la integración frente a coordinar una respuesta asíncrona entre Django y una Lambda.
Código de seguimiento UUID	Funciona como credencial de un solo ticket: no se puede adivinar ni enumerar, y permite el seguimiento público sin exponer ningún otro dato del sistema.

Resiliencia y observabilidad

Las dependencias externas (AWS y la API de IA) se tratan como "best effort": si fallan, la operación principal del usuario nunca se bloquea. El envío a SQS está protegido con manejo de errores y timeouts cortos, y la llamada a Gemini corre con un tope de 8 segundos y devuelve prioridad Media si no hay respuesta. La observabilidad se apoya en CloudWatch para la Lambda y en los logs de la aplicación en Render.

4. Calidad y control de versiones

- **Pruebas:** Suite de tests automatizados que cubre autenticación, permisos, tickets, comentarios, historial, filtros y paginación, además de las notificaciones (con la llamada a AWS simulada), la prioridad con IA y el reporte público.
- **Git:** Historial de Git con commits descriptivos bajo la convención Conventional Commits, agrupados por unidad de trabajo (modelo, lógica, interfaz y tests).
- **Documentación:** El archivo AI-DECISIONS.md, en la raíz del repositorio, registra el problema, el prompt, la propuesta de la IA y la validación o corrección humana de las decisiones asistidas.

5. Conclusiones

TicketLite Cloud cumple el objetivo de resolver un problema cotidiano de los equipos chicos con una arquitectura simple y desacoplada: la aplicación no espera a que se envíe un email para responder, y la caída de un servicio externo, ya sea AWS o la API de IA, no impide crear ni consultar tickets. El resultado es un sistema que funciona en producción, con un flujo interno para el equipo y un canal público para quienes reportan sin tener cuenta.

El desarrollo dejó aprendizajes concretos sobre cómo se comporta un sistema que depende de servicios externos: la importancia de acotar los tiempos de espera, el hecho de que los modelos de IA se retiran y hay que poder cambiarlos sin tocar el código, y las diferencias entre el entorno local y el de producción, que obligaron a revisar variables de entorno, dependencias y migraciones en el deploy.